

Smart Home Energy Consumption Predictor

Project Overview — what it is, what it is for, and how it works

1. What is this?

An end-to-end Python project that turns raw smart-home sensor readings into insight and a forecast. It **loads, cleans, explores and predicts** household appliance energy use, and presents everything in a single interactive web dashboard. Energy is recorded every 10 minutes from appliances, lighting, indoor room sensors (temperature and humidity) and an outdoor weather station.

2. What is it used for?

It answers practical questions about a home's electricity use and helps anticipate it:

- **Understand consumption** — when energy is used across the day and week, and how usage splits into low / medium / high periods.
- **Spot the drivers** — which conditions (time of day, lighting, indoor and outdoor temperature) move energy use the most.
- **Predict usage** — estimate appliance energy for a given set of conditions, and explore “what-if” scenarios live.
- **Support decisions** — shift loads to cheaper / greener hours, size batteries or solar, flag abnormal consumption, and inform efficiency or demand-response programmes.

Typical users: homeowners and smart-home enthusiasts, energy analysts and utilities, and students or developers learning an end-to-end data + machine learning workflow.

3. How it works

The project is split into small, single-purpose modules that form a clear pipeline: **generate** → **load** → **clean** → **engineer features** → **visualise** → **predict**.

3.1 Data

On first run the project generates a realistic dataset of ~19,700 readings (10-minute intervals over several months). The data is behaviour-driven: appliance energy follows daily routines (morning and evening peaks), a weekend bump, and a per-day “household activity” level, with a weak weather influence and occasional high-usage spikes — so daily averages vary naturally rather than following any artificial trend. You can drop in your own CSV of the same layout to use real readings instead.

3.2 Clean & feature engineering

- Parse timestamps, sort chronologically, remove duplicate readings.
- Drop non-predictive random columns; fill any sensor gaps with time-aware interpolation.
- Add calendar features (hour, day of week, month, weekend flag, time-of-day), aggregates (total energy, average indoor temperature and humidity) and a Low / Medium / High *energy tier*.

3.3 Visualise — interactive dashboard

A Streamlit web app presents the data across four tabs, with sidebar filters (date range, days of week, time of day) that update every chart and KPI live. All charts are interactive Plotly graphs — hover for values, zoom, pan and toggle series:

- **Overview** — daily trend with 7-day rolling average, usage-tier donut, average energy by hour.
- **Usage Patterns** — hour × day-of-week heatmap, time-of-day distributions, weekly appliances vs lights.
- **Correlations** — correlation matrix, top energy drivers, outdoor-temperature vs energy scatter.
- **Predictor** — model accuracy, feature importance, actual vs predicted, and a live what-if simulator.

3.4 Predict — machine learning model

A **Random Forest regressor** (scikit-learn) predicts appliance energy (in Wh) from the temperature, humidity, weather and time features. It is trained on a portion of the data and scored on a held-out test set, reporting **R²**, **MAE** and **RMSE**. A feature-importance ranking shows which inputs matter most (time of day dominates), and the **what-if simulator** lets you set the hour, day type, temperatures and lighting to get an instant energy estimate.

4. Project structure

app.py	Interactive Streamlit dashboard (the main deliverable).
data_loader.py	Load → clean → feature-engineer pipeline.
generate_sample_data.py	Realistic dataset generator.
model.py	Random Forest predictor, metrics and what-if helper.
config.py	Central paths, theme colours and column groups.
requirements.txt	Python dependencies.
data/energy_data.csv	The dataset (auto-generated; replaceable).

5. Tech stack

Python • **pandas** / **NumPy** (data) • **scikit-learn** (model) • **Plotly** (interactive charts) • **Streamlit** (web dashboard).

6. How to run

Install the dependencies and launch the dashboard — it opens in the browser and generates its data automatically on first run:

```
pip install -r requirements.txt
streamlit run app.py
```

For a permanent shareable link, the app can be deployed from its GitHub repository to Streamlit Community Cloud, which serves it at a public*streamlit.app* URL.